

AI-val gyorsabban termelünk kódot. De ki fogja átnézni?

Mit mutat az agentic coding magyar IT-vitája a fejlesztés valódi szűk keresztmetszeteiről?

Kutatási jegyzet és teljes cikk · forrásalapú stratégiai elemzés

Vezérgondolat. Az agentic coding nem a szoftverfejlesztés végét hozza, hanem a felelősségi lánc újratárgyalását. A kódgenerálás drámaian olcsóbb lett, de a megértés, a validáció, az architektúra, a security és az ownership nem. Ezért a magyar IT-ban nem AI-pánik a fő történet, hanem az, hogy a kódírás gyorsulása láthatóvá teszi, hol volt eddig is gyenge a mérnöki kultúra. Az AI ott gyorsító, ahol van specifikáció, review, teszt és tulajdonlás; ahol ezek hiányoznak, ott nem megoldja a problémát, hanem gyorsabban termeli láthatóvá. A kérdés tehát nem az, hogy használjunk-e coding agenteket, hanem hogy milyen mérnöki kontroll mellett.

Címjavaslatok

1. AI-val gyorsabban termelünk kódot. De ki fogja átnézni? **(kiválasztva)**
2. Nem a kód volt a szűk keresztmetszet: mit mutat a magyar IT vitája az AI-fejlesztésről?
3. Az AI nem váltotta ki a senior fejlesztőt. Csak átterelte hozzá a felelősséget.
4. Agentic coding Magyarországon: gyorsulás vagy review-válság?
5. A fék lekerült a kódtermelésről. A felelősségről nem.

Miért a legerősebb: egyetlen mondatban kérdezi meg a cikk tézisé (a kódtermelés gyorsulása nem ér semmit a review-, validációs- és ownership-kapacitás nélkül), azonnal megfogja a konfliktust, és nem clickbait, hanem állítás. A „ki fogja átnézni” kérdés végigvihető a teljes szövegen, a leadtől a zárásig.

I. rész - Kutatási jegyzet

Jelölések: [tény] = mért, megtörtént adat · [becslés/előrejelzés] = modellezett vagy előrejelzett adat · [vélemény] = szakértői vagy piaci álláspont · [kvalitatív jelzés] = Reddit/thread/fórum alapú megfigyelés, nem reprezentatív adat. Ahol egy forrás kis mintás, megfigyeléses vagy vendor-eredetű, azt külön jelezzük.

1. A tíz legfontosabb adatpont

#	Adat	Forrás, dátum, típus	Értelmezés
1	90% AI-használat a fejlesztésben, egy év alatt +14 százalékpont.	DORA / Google, 2025 · ~5 000 fő survey + 100+ óra kvalitatív · [tény]	Az adoptáció eldőlt. A kérdés már nem az, hogy használjuk-e, hanem milyen kontroll mellett.
2	84% használ vagy tervez használni AI-eszközt; 46% NEM bízik a pontosságban (2024: 31%); csak 3% bízik erősen.	Stack Overflow Survey, 2025 · 49 009 fő · [tény]	Az adoptáció-bizalom olló a 2025 legbeszédesebb mintázata: nő a használat, csökken a bizalom.
3	85% rendszeresen használ AI-t; 62% coding assistantre/agentre támaszkodik; 90% legalább heti 1 órát spórol.	JetBrains, 2025 · 24 534 fő · [tény + önbevallás]	Az érzékelt haszon magas - de ez önbevallás, nem mért szállítási teljesítmény.
4	Havi 43,2M merged PR (+23% év/év); ~1 milliárd commit (+25%); a Copilot-agent 5 hónap alatt 1M+ PR-t nyitott.	GitHub Octoverse, 2025 · platformaktivitás · [tény]	A „PR-gyár” élmény mérhető volumenrobbanás, nem szubjektív panasz - de volument mér, nem értéket.
5	82% szerint az AI gyorsítja a kódolást; 95% review-za/javítja az AI-outputot; csak 48% ellenőrzi mindig commit előtt.	Sonar State of Code, 2026 · 1 149 fő · [tény + vendor]	A review beépült a munkába, de egyenletlenül: az ellenőrzés nem garantált.
6	Tapasztalt fejlesztők érett kódbázison 19%-kal LASSABBAK lettek AI-val; előtte +24% gyorsulást vártak.	METR RCT, 2025.07 (arXiv:2507.09089) · n=16 · [tény - egy konkrét RCT]	Percepció-valóság szakadék. Kis minta, speciális kontextus; a hatás kontextusfüggő.
7	Izolált, körülhatárolt feladaton Copilottal 55,8% GYORSULÁS.	Peng és tsai, 2023 (GitHub RCT) · [tény]	Izolált feladaton az AI mérhetően gyorsít - de ez nem a teljes feature-átfutás.
8	Core fejlesztők: +6,5% review, -19% saját kódproduktivitás. Perifériális fejlesztők: +43,5% commit.	Xu és tsai (Tilburg), 2025 · megfigyeléses, 2 755 projekt · [tény - megfigyeléses]	A teher a szeniorokra toódik: a junior termel, a senior review-zik és takarít.
9	7 703 AI-fájlból 87,9% nincs CWE-sérülékenység, de 4 241 instancia 77 típusban; kontrollált benchmarkon csak 55% biztonságos.	Schreiber & Tippe 2025; Veracode 2025 · [tény]	A biztonsági kockázat valós, nyelv- és kontextusfüggő, és review nélkül halmozódik - de nem „általában lyukas”.
10	A piacot a szenior pozíciók dominálják; a túlterhelt szeniorok kiegészése és fluktuációja külön kockázat; stagnáló IT-bérek.	Hays Hungary Salary Guide, 2025/26 · 4 200+ fő · [tény, de általános HU munkaerőpiac]	Magyar kontextusjelző, nem AI-bizonyíték: épp a legszűkösebb réteg kerül nyomás alá.

2. A tíz legfontosabb szakmai törésvonal

#	Törésvonal	Állítás	Mire épül?	Értékelés
1	Kódírás vs szoftverszállítás	A gépelés gyorsul, de a szállítás nem feltétlenül.	METR; Sonar; magyar fórum	ERŐS
2	Lokális vs rendszer-produktivitás	A lokális gyorsulás value stream management nélkül downstream káoszba fordul.	DORA 2025	ERŐS (irány) / KÖZEPES (mérték)
3	AI-adoptáció vs AI-bizalom	Nő a használat, miközben csökken a bizalom a pontosságban.	Stack Overflow; Sonar	ERŐS
4	Izolált feladat vs érett kódbázis	A hatás kontextusfüggő: +55,8% vs -19%.	Peng; METR	ERŐS (kontextus) / METR kis minta
5	PR-volumen vs review-kapacitás	A PR-termelés gyorsabban nő, mint a review-kapacitás.	Octoverse; Sonar; fórum	KÖZEPES-ERŐS
6	Vibe coding vs AI-assisted engineering	A kritika célpontja a vibe coding, nem maga az AI.	SO (72% nem vibe-codol); fórum	KÖZEPES (definíció, kvalitatív)
7	Junior output vs senior review-teher	A junior termelékenység nő, a teher a szeniorokra száll.	Tilburg/Xu	KÖZEPES (megfigyelés, OSS)
8	Automation vs ownership	A generálás automatizálható, a felelősség nem.	Sonar (48% ellenőriz); koncepció	KÖZEPES (részben normatív)
9	Security-kockázat vs mérhető kontroll	A kockázat valós, de nyelv-/kontextusfüggő és kezelhető.	Schreiber & Tippe; Veracode	KÖZEPES (mérési módszertől függ)
10	Magyar vékony QA/platform vs gyorsítási nyomás	A strukturális gyengeségek felerősítik a problémát.	Hays + fórum; hipotézis	GYENGE-KÖZEPES (kevés HU adat)

3. A magyar agentic coding vita valóságtesztje

Állítás / félelem	Alátámasztó bizonyíték	Gyengítő vagy árnyaló bizonyíték	Értékelés
„Elszabadult a PR-gyár”	Octoverse volumen: 43,2M PR/hó, 1M+ agent-PR; magyar fórum.	A volumen nem minőség; a PR-szám felfújható.	ERŐS (volumen)
„A fejlesztés gyorsult, de a szállítás nem”	METR; Sonar review-teher; fórum.	Izolált feladaton gyorsul (Peng); magas érzékelt haszon.	KÖZEPES-ERŐS, kontextusfüggő
„Az AI több tech debtet termel”	Tilburg: a rework nő az AI-adopció után.	Megfigyeléses, OSS-kontextus; jó workflow-val csökkenthető.	KÖZEPES
„A review lesz az új bottleneck”	Tilburg +6,5% review; Sonar 95% review-zik; SO 45% debug tovább tart.	Csak 48% ellenőriz mindig - sok helyen NEM a review a szűkület (ami önmagában aggasztó).	ERŐS
„A gyengébb fejlesztők AI-val profibbnak látszanak”	Fórum (kvalitatív); Tilburg: perifériális +43,5% commit.	Nincs reprezentatív mérés a „látszat”-ra; közvetett.	KÖZEPES (kvalitatív)
„A senior szerepe felügyeleté alakul”	Vella & Blincoe: 82% kevesebb kódírás, supervisory work.	Kis, önbevalláson alapuló minta.	KÖZEPES-ERŐS
„Az agent workflow megoldja a problémát”	Anekdotikus jó setupok a fórumon.	Nincs reprezentatív bizonyíték; a workflow segít, de nem pótol governance-t.	GYENGE (túlzó)
„A magyar IT-ban ez különösen fájni fog”	Hays: szenior kiégés; strukturális hipotézis.	Nincs reprezentatív magyar agentic-adat; hipotézis.	KONTEXTUSFÜGGŐ

4. Cikkvázlat

- Lead - a PR-gyár élménye egy reviewer szemszögéből, azonnal a GitHub Octoverse volumenadataira kötve.
- 1. Mi változott meg valójában? - a coding agent nem autocomplete, hanem felügyelt junior-rendszer.
- 2. Mi az, ami már most is látszik? - adoptáció, bizalom és volumen narratívába építve, nem táblázatban.
- 3. Hol kezdődik a félreértés? - a kódírás nem egyenlő a szoftverszállítással; a négy fogalom; Peng vs METR.
- 4. A magyar Reddit mint kvalitatív terepnapló - törésvonalak és visszatérő figurák.
- 5. Vibe coding vs AI-assisted engineering - éles különbségtétel.
- 6. Az új senioritás - a felügyeleti és validációs mérnöki munka felértékelődése.
- 7. Miért fájhat ez különösen a magyar IT-ban? - óvatos hipotézis a Hays-kontextussal.
- 8. Mit kellene mérni? + Zárás - mérési fegyelem és stratégiai tanulság.

II. rész - A teljes cikk

AI-val gyorsabban termelünk kódot. De ki fogja átnézni?

Mit mutat az agentic coding magyar IT-vitája a fejlesztés valódi szűk keresztmetszeteiről?

Egy platform engineer tavaly még napi két-három pull requestet nézett át egy közepes magyar csapatnál. Idén tízet. A változások egy része ezer sor körüli, néhol ott a ticketen kívüli, „amúgy is itt jártam” módosítás. A CI néha alig bírja, a review-sor folyamatosan tele, és a kódbázis lassan olyan, mintha többen írták volna, mint ahányan valójában dolgoznak rajta. A csapat nem lett kétszer akkora. A coding agent viszont megérkezett.

Könnyű ezt egyéni panaszként elintézni. Pedig nem az. A GitHub 2025-ös Octoverse-jelentése szerint a platformon havonta átlagosan 43,2 millió pull request olvad be - ami 23%-os éves növekedés -, és 2025-ben közel egymilliárd commit született. Csak a Copilot kódoló ügynöke öt hónap alatt több mint egymillió PR-t nyitott. A „PR-gyár” érzete tehát nem hangulat, hanem egy globális volumenrobbanás helyi lecsapódása. A kérdés, amit ez a cikk feltesz, ennyi: ha a kódírás öt-tízszer gyorsabb lett, ki fogja öt-tízszer gyorsabban átnézni, megérteni és vállalni, amit legenerálunk?

1. Mi változott meg valójában?

Érdemes tisztázni, mi a coding agent, mert a köznyelvben összemosódik a régi autocomplete-tel. A mai agent már nem csak sort egészít ki: tervet készít, több fájlt módosít, tesztet ír, pull requestet nyit, alügynököket indít, és előre beállított ellenőrzéseket - hookokat - futtat. Ez minőségileg más, mint két éve volt.

Ettől viszont nem lett autonóm szenior mérnök. Sokkal pontosabb úgy gondolni rá, mint egy gyors, olcsó és fáradhatatlan junior-rendszerre: rengeteget termel, gyakran használható, de a kontextust nem érti úgy, mint aki a rendszert évek óta építi, és a felelősséget sem tudja viselni. Akkor hasznos, ha van fölötte mérnöki kontroll - specifikáció, határok, review. Magára hagyva pont azt csinálja, amit egy felügyelet nélküli junior: sokat, gyorsan, és néha magabiztosan rossz irányba. A magabiztosság itt a kockázat: az agent ritkán jelzi, ha bizonytalan, és olyan kódot is meggyőzően ír, ami szintaktikailag hibátlan, de a rendszer logikájába nem illik. Ezt jellemzően csak az veszi észre, aki a rendszert valóban érti.

2. Mi az, ami már most is látszik?

Az AI-használat kérdése eldőlt. A DORA 2025-ös, közel ötezer szakembert megkérdező kutatása szerint a fejlesztők 90%-a használ AI-t, ami egy év alatt tizennégy százalékpontos ugrás. A JetBrains huszonnégyezres felmérésében 85% rendszeresen használja, és 62% legalább egy kódoló asszisztensre vagy ügynökre támaszkodik. A Stack Overflow szerint 84% használ vagy tervez használni AI-eszközt, az új GitHub-fejlesztőknek pedig 80%-a már az első héten elkezdte. Itt nincs több vita: a kérdés már nem az, hogy használjuk-e, hanem az, milyen szervezeti kontroll mellett.

Mert a használat mellett egy másik szám is makacsul tartja magát - a bizalomé, ami csökken. A Stack Overflow-nál a fejlesztők 46%-a kifejezetten nem bízik az AI-kimenet pontosságában; egy éve ez még 31% volt, és mindössze 3% bízik benne erősen. A Sonar 2026-os, több mint ezer fős kutatásában a fejlesztők 96%-a nem bízik teljesen abban, hogy az AI-kód funkcionálisan helyes. Ez a kettősség - magas használat, alacsony bizalom - a 2025-ös év legbeszédesebb mintázata, és nem technofóbia: egy determinista gondolkodású szakma józan reakciója egy nemdeterminista eszközre. A fejlesztők ráadásul nagyrészt hasznosnak is érzik az AI-t; a Sonar szerint 82% egyetért, hogy gyorsítja a kódolást, a JetBrains szerint 90% legalább heti egy órát spórol vele. Ezek valós érzetek - és pontosan itt kezdődik a félreértés.

3. Hol kezdődik a félreértés?

A baj abból fakad, hogy a kódírás sebességét összekeverjük a szoftverszállítás sebességével. Pedig négy különböző dolog ez, amit a hétköznapi beszéd egybeemos: hogy milyen gyorsan keletkezik a kód, hogy mennyire érzi gyorsabbnak magát a fejlesztő, hogy mennyi idő alatt jut el egy igény a felhasználóig, és hogy mennyire tartható karban, amit építettünk. Az AI az első kettőt szinte mindig javítja. A harmadikat és a negyediket nem garantálja. A felbontás lényege egy költség-aszimmetria: a generálás olcsó lett, a megértés nem. Egy ezer soros, géppel írt PR átolvasása, fejben tartása és felelős jóváhagyása ugyanannyi - vagy több - kognitív munkát kíván, mint korábban; csak ma nem feltétlenül az végzi, aki a kódot „írta”, hanem a reviewer.

Ezt a szakadékot két kutatás mutatja meg a legélesebben, egymással szemben. Egy korai, izolált feladatra épülő GitHub-kísérletben a Copilotot használók 55,8%-kal gyorsabban végeztek - ez a „gyorsít” tábor emblematikus száma. A METR 2025-ös randomizált kontrollált kísérletében viszont tizenhat tapasztalt nyílt forrású fejlesztő dolgozott a saját, érett kódbázisain, és AI-val 19%-kal lassabban végeztek - miközben előzetesen 24% gyorsulást vártak, és utólag is 20%-ot éreztek. A két eredmény nem mond ellent egymásnak; különböző dolgot mér. A tanulság nem az, hogy „az AI gyorsít”, és nem is az, hogy „az AI lassít”, hanem az, hogy a hatás kontextusfüggő: izolált, jól körülhatárolt feladaton jellemzően gyorsít, nagy, jól ismert kódbázison, szigorú minőségi elvárás mellett viszont a takarítás költsége felfalhatja a generálás nyereségét. A METR kis mintás volt és korai modelleket használt, a szerzők maguk is pillanatfelvételnak nevezik - de épp a percepció és a valóság közötti rés a fontos: még a saját gyorsulásunkat is rosszul becsüljük.

4. A magyar Reddit mint kvalitatív terepnapló

Itthon ez a feszültség a fejlesztői fórumokon csapódik le a legélesebben. Fontos azonban tisztázni: ezek a threadek nem reprezentatív felmérések. Reprezentatív magyar adat az agentic coding hazai elterjedtségéről egyelőre alig van, ezért a fórumhangokat kvalitatív terepnaplóként kezelem, nem statisztikai bizonyítékként.

Jelzésként viszont pontosak, mert ugyanazok a figurák és ugyanaz a törésvonal tér vissza bennük. Ott a platform engineer, akire ráomlik a PR-volumen; a senior reviewer, aki napokat tölt mások által generált, ránézésre rendben lévő, de gyengén megértett kód olvasásával; a termékmenedzser, aki látja, hogy az üzlet a mennyiséget díjazza; az infra engineer, akinek az AI valóban segít a scriptelésben, miközben mások elhamarkodott infra-módosításait takarítja; az AI-native fejlesztő, aki jól használja, mert tud specifikálni és ellenőrizni; és a vibe coder, aki bedob egy ticketet, és felpushol egy nagy PR-t. A konfliktus nem fejlesztők kontra menedzsment. A konfliktus a sebesség és a minőség, az üzleti nyomás és a mérnöki fegyelem, a látszólagos és a valódi haladás között feszül.

Van ennek egy kényelmetlen, szinte szociológiai rétege is. A precíz fejlesztő, aki valóban érti és validálja a munkát, lassabbnak látszik - mert tényleg lassabb annál, aki csak felpushol. A gyengébb fejlesztő AI-val ránézésre jobb outputot produkál. Nehezebb lett első ránézésre megmondani, ki ért hozzá valójában. Ez nemcsak minőségi, hanem bizalmi és státuszkérdés is egy csapaton belül.

5. Vibe coding és AI-assisted engineering

Itt érdemes egy különbséget élesen meghúzni, mert a magyar vita nagy része feloldható vele. A fórumokon a legtöbben nem az AI-t támadják, hanem azt, amikor az AI-val megspórolják a gondolkodást.

Az egyik véglet a vibe coding: egy ticket, egy Figma-kép vagy egy laza ötlet bemásolása a modellbe, nagy scope, kevés megértés, nagy PR, ticketen kívüli módosítások, „zöld a teszt, mehet” hozzáállás és elkent felelősség. A másik az AI-assisted engineering: előzetes specifikáció és engineering design, kis scope, korlátozott agent-jogosultság, karbantartott kontextusfájlok (CLAUDE.md, AGENTS.md, ai-docs), lintet, tesztet, típus- és security-ellenőrzést futtató hookok, AI-review plusz emberi review, és egyértelmű emberi felelősség. Ugyanaz a modell, ugyanaz az eszköz - két gyökeresen különböző mérnöki kultúra.

Megnyugtató adat egyébként, hogy a Stack Overflow szerint a fejlesztők 72%-a a saját professzionális munkájában nem vibe-codol: a jelenség hangosabb, mint amilyen elterjedt.

6. Az új senioritás

Ha az adatokat egyetlen mondatba kellene sűríteni, az így szólna: a senioritás nem szűnik meg, hanem áthelyeződik. Egy 2026-os longitudinális vizsgálat szerint a fejlesztők 82%-a kevesebb időt tölt kódírással, és a munka súlypontja a létrehozás felől a validáció felé tolódik - a szerzők ezt supervisory engineering worknek, felügyeleti mérnöki munkának nevezik. A minta kicsi és önbevalláson alapul, de az irány egybevág a többi forrással.

A jó szenior egyre kevésbé az, aki kézzel írja a legtöbb sort. Inkább az, aki specifikál, határokat húz, architektúrát véd, review-zik, validál, üzleti logikát fordít, security- és observability-szemponthoz kér számon, és - talán a legfontosabb - eldönti, mit nem szabad az AI-ra bízni. A kézi kódolás presztízse csökkenhet, de a mérnöki ítélőképesség értéke nő. A jó munkamenet nem bonyolult: specifikáció, releváns kontextus, kis lépés, AI-implementáció, AI-review, emberi review, teszt, merge. Ebben a modellben az agent a gyors, de felügyelt junior - a végső döntés és a felelősség emberé marad. És épp ez a réteg az, ami nem automatizálódik el egyhamar: a modell és a rendszer közötti fordítás, az üzleti szándék lekódolása, és annak eldöntése, mit jelent egyáltalán, hogy valami „kész”.

7. Miért fájhat ez különösen a magyar IT-ban?

Ezt óvatos hipotézisként mondom, nem ítéletként, mert reprezentatív magyar adat itt is hiányzik. Több strukturális ok miatt üthet ez nálunk élesebben.

Sok magyar csapatban eleve vékony a platform-, QA-, security- és architektúra-réteg, a szeniorok pedig már az AI előtt is túlterheltek voltak. Ha erre ráengedünk egy agentic coding munkamenetet megfelelő korlátok nélkül, az nem felszabadítja a szeniorokat, hanem review-pokollá teszi a napjaikat. A kisebb cégeknél gyakran hiányzik a formális engineering governance; a multiknál és a szolgáltatóközpontoknál viszont fragmentált a tulajdonlás, sok az approval-réteg és az offshore-nearshore interfész. A munkaerőpiaci bizonytalanságban ráadásul nehezebb visszadobni egy rossz PR-t vagy nemet mondani egy irreális határidőre. És itt válik kínosan láthatóvá valami, ami eddig is így volt: sok helyen a review volt a rejtett minőségbiztosítás - az a pont, ahol egy tapasztalt ember csendben kijavította, amit a folyamat elrontott. A kockázat ráadásul szegmensfüggő: egy korai fázisú startupnál a gyors iteráció valódi előny, és a slop ára alacsony, mert a kód úgyis gyorsan cserélődik; egy banki vagy adatnehéz rendszerben viszont ugyanaz a slop évekig elven, és sokszor csak élesben, incidensként derül ki.

Kontextusjelzőként ide kívánczik a Hays magyar munkaerőpiaci jelentése, amely a szenior szakemberek túlterhelését, kiégését és az ebből fakadó fluktuációt külön kockázatként nevezi meg, stagnáló IT-bérek mellett. Ez nem reprezentatív bizonyíték az agentic coding magyar hatására - de jól mutatja, hogy épp az a réteg kerül nyomás alá, amelyből amúgy is kevés van, és amelyet a legdrágább pótolni.

8. Mit kellene mérni?

A megoldás nem AI-politikai, hanem engineering-politikai. És a kiindulópont a mérés: ha az AI-használatot a leírt kódsorokban, a PR-ok számában vagy a lezárt ticketekben mérjük, akkor olyan mutatókat figyelünk, amelyeket az AI-val triviálisan fel lehet fújni anélkül, hogy egyetlen valódi probléma is megoldódna.

Mit mérjen egy csapat a LOC és a PR-szám helyett?

Cycle time - az ötlettől a production deployig eltelt idő.

Change failure rate - a deployok hány százaléka okoz incidenst.

Escaped defects - a productionben kibukó hibák száma.

Review lead time és a review-terhelés reviewerenként.
A PR-méret eloszlása és a „túl nagy” PR-ok aránya.
Rework / churn rate - a frissen írt kód mekkora hányadát írják át rövid időn belül.
Incident count és MTTR (átlagos helyreállási idő).
Tesztlefedettség és annak változása az AI-bevezetés óta.
A security findingok száma és súlyossága.
Az out-of-scope (ticketen kívüli) változtatások aránya.

A logika egyszerű: a produktívismerés egy részét vissza kell forgatni minőségbe és tesztlefedettségbe, és aki AI-val generál kódot, ugyanúgy felelős érte, mintha kézzel írta volna.

9. Zárás

Az agentic coding nem azt a kérdést teszi fel, hogy kell-e még fejlesztő. Azt teszi fel, hogy volt-e valaha valódi mérnöki kultúra a kód körül.

A kódgenerálás drámaian olcsóbb lett. A megértés, a validáció, az architektúra, a security és a tulajdonlás nem. Ahol van specifikáció, review, teszt, ownership és mérési fegyelem, ott az AI gyorsító, amely felszabadítja a seniorokat az ismétlődő munka alól. Ahol ezek hiányoznak, ott az AI nem oldja meg a problémát - csak gyorsabban termeli láthatóvá. A különbség nem a modellben van, hanem a szervezetben, amelyik használja. És ezt a különbséget, ha eddig el is lehetett fedni, mostantól egyre nehezebb lesz.

III. rész - Felhasznált források

Elsődleges / nagy megbízhatóságú források

1. DORA / Google Cloud - State of AI-assisted Software Development 2025 (~5 000 fős globális survey + 100+ óra kvalitatív adat).
2. METR - Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity (arXiv:2507.09089, 2025.07; 2026.02-i módszertani frissítés). RCT, n=16.
3. Xu, F., Medappa, P. K., Tunc, M. M., Vroegindeweij, M., Fransoo, J. C. (Tilburg) - AI-Assisted Programming Decreases the Productivity of Experienced Developers by Increasing the Technical Debt and Maintenance Burden (arXiv:2510.10165 / SSRN, 2025). Megfigyeléses, 2 755 projekt, 1 699 fejlesztő.
4. Schreiber, M. & Tippe, P. - Security Vulnerabilities in AI-Generated Code: A Large-Scale Analysis of Public GitHub Repositories (ICICS 2025; arXiv:2510.26103). 7 703 fájl, CodeQL.
5. Peng, S., Kalliamvakou, E., Cihon, P., Demirel, M. - The Impact of AI on Developer Productivity: Evidence from GitHub Copilot (arXiv:2302.06590, 2023). RCT.
6. Vella, A. & Blincoe, K. - longitudinális vizsgálat a fejlesztői munka eltolódásáról („supervisory engineering work”), arXiv, 2026. 95 párosított résztvevő.

Megbízható másodlagos források

1. Stack Overflow - Developer Survey 2025 (49 009 fő, 166+ ország).
2. JetBrains - The State of Developer Ecosystem 2025 (24 534 fő, 194 ország).
3. GitHub - Octoverse 2025 (platformaktivitási adatok: PR-ok, commitok, Copilot-agent).
4. Sonar - State of Code Developer Survey 2026 (1 149 fő; vendor-kutatás).
5. Veracode - 2025 GenAI Code Security Report (100+ LLM, 80 kódfeladat; vendor-kutatás).
6. Hays Hungary - Salary Guide 2025/2026 (4 200+ válaszadó; általános magyar munkaerőpiac).
7. Korábbi biztonsági vizsgálatok: Pearce és tsai (2022); Fu és tsai (2023) - kontrollált CWE-elemzések.

Magyar kvalitatív források / fórumanyagok

1. r/programmingHungary threadek (a coding agentek okozta gyorsulásról, az agent-first setupokról, a felelős fejlesztői munka nehézségeiről és a bedőlő AI-projektekről). Kvalitatív terepanyag, nem reprezentatív adat.

Forráskezelési megjegyzés. Minden számszerű állításhoz tartozik forrás. A kutatási jegyzet külön jelöli, mi [tény], mi [becslés/előrejelzés], mi [vélemény] és mi [kvalitatív jelzés]. A magyar Reddit-threadek kvalitatív terepanyagként szerepelnek, nem reprezentatív adatként. Ahol egy forrás kis mintás (METR, n=16), megfigyeléses (Tilburg) vagy vendor-eredetű (Sonar, Veracode), ott ezt kifejezetten jelezzük. A konkrét százalékok gyorsabban avulnak, mint a következtetések iránya - a terület 2025 eleje óta jelentősen fejlődött.

Függelék - címek és LinkedIn hookok

5 alternatív cím

1. Nem a kód volt a szűk keresztmetszet: mit mutat a magyar IT vitája az AI-fejlesztésről?
2. Az AI nem váltotta ki a senior fejlesztőt. Csak áttérte hozzá a felelősséget.
3. Agentic coding Magyarországon: gyorsulás vagy review-válság?
4. A vibe coding ára: hogyan változtatják meg a coding agentek a magyar fejlesztői munkát?
5. A fék lekerült a kódtermelésről. A felelősségről nem.

5 LinkedIn-poszt hook

1. „Gyorsabban fejlesztünk, de nem szállítunk gyorsabban.” Egy mondat, ami pontosan leírja, mit csinált az agentic coding a csapatokkal. (thread)
2. A DORA szerint a fejlesztők 90%-a használ AI-t - a Stack Overflow szerint viszont 46% nem bízik a pontosságában. Az adoptáció-bizalom olló a 2025-ös igazi sztori.
3. Egy RCT-ben tapasztalt fejlesztők 19%-kal lassultak az AI-tól - miközben gyorsabbnak érezték magukat. Nem az AI a hibás, hanem amit az érzékelt produktivitásból következtetünk.
4. Az AI nem tünteti el a mérnöki munkát. Áthelyezi: kevesebb kódírás, több review, validáció és specifikáció. A senior értéke az lett, hogy tudja, mit NEM szabad az AI-ra bízni.
5. Ha LOC-ban vagy PR-számban méred az AI-produktivitást, AI-val bármelyik szám felfújható. Itt van 10 metrika, amit helyette érdemes nézni. (carousel/thread)